Mini-Pixels HTTP 流存储读写性能优化实验报告

罗呈语

2026 年 10 月 26 日

1 实验背景与目标

PixelsDB 是一个分布式数据库系统，节点间数据传输依赖 HTTP 协议。本次实验针对 `mini-storage-http` 模块中的 `HttpInputStream` 与 `HttpOutputStream` 进行性能优化。原实现存在忙等待、队列锁操作冗余、内存无界增长等问题，在高吞吐场景下会导致 CPU 空转、延迟增加甚至内存溢出。本实验的目标是在不改变外部接口的前提下，消除确定性开销、引入背压机制，并修复潜在的内存泄漏。

2 实验环境

- 操作系统：Ubuntu 24.04（WSL2）
- JDK：OpenJDK 21
- 构建工具：Maven 3.x，Surefire 3.2.5
- 项目：`mini-pixels`，模块 `mini-storage-http`
- 测试类：`TestHttpStream`

3 原始代码问题分析

3.1 HttpInputStream

1. **忙等待**：`emptyData()` 使用 `Thread.sleep(DELAY_MS)` 轮询队列，即使数据在睡眠期间到达也必须等待睡眠结束，浪费 CPU 并增加延迟。
2. **队列操作冗余**：`read()` 与 `read(byte[],int,int)` 每轮循环先 `peek()` 再在耗尽后 `poll()`，每个 ByteBuf 触发两次加锁操作。
3. **队列类型低效**：`LinkedBlockingDeque` 支持双端操作，但本场景仅需 FIFO，双端能力带来额外开销。
4. **内存泄漏**：`close()` 未释放队列中残留的 `ByteBuf` 与当前正在消费的 `ByteBuf`，Netty 直接内存可能泄漏。

3.2 HttpOutputStream

1. **无界队列导致 OOM**：`contentQueue` 为无界 `LinkedBlockingQueue`，当发送速度低于写入速度时，队列无限增长，最终耗尽堆内存。这是实验中 `OutOfMemoryError` 的直接原因。
2. **频繁数组复制**：`flushBufferAndRewind()` 与 `flush()` 每次使用 `Arrays.copyOfRange` 生成新数组，产生大量临时垃圾。
3. **缺少背压**：写入端不感知发送端进度，无法自适应降速。

4 优化方案

4.1 HttpInputStream 优化

- 将 `emptyData()` 的 `Thread.sleep` 轮询替换为 `contentQueue.poll(timeout, NANOSECONDS)`，保留原 `MAX_TRIES × DELAY_MS` 的总超时语义。
- 新增 `currentBuf` 字段持有当前 `ByteBuf`,每个块只执行一次 `poll()`,消除 `peek()+poll()` 双重操作。
- 队列类型改为 `LinkedBlockingQueue`，利用 put/take 分离锁降低生产者-消费者锁竞争。
- `close()` 中调用 `releaseResources()`，释放 `currentBuf` 并排空队列中所有 `ByteBuf`。
- 修正 `StreamHttpServerHandler` 中 header 判断的潜在 NPE（常量前置）。

4.2 HttpOutputStream 优化

- 将 `contentQueue` 改为有界 `ArrayBlockingQueue(QUEUE_CAPACITY)`，容量设为 16。队列满时 `put()` 阻塞写入线程，形成背压，防止内存无限增长。
- `flush()` 与 `flushBufferAndRewind()` 使用 `putContent()` 封装 `put()`,将 `InterruptedException` 转为 `IOException`。
- `close()` 发送结束标记也使用 `put()`，确保结束标记在队列有空间时入队。
- 保留原有重试与连接复用逻辑，未改变 HTTP 语义。

5 预期效果与定性分析

- **CPU 占用下降**：忙等待消除后，接收端在无数据时进入阻塞等待，不再周期性唤醒。
- **接收吞吐提升**：减少队列锁操作次数，尤其在大块连续读取时效果明显。
- **内存稳定**:发送端有界队列形成背压,写入速度自动匹配发送速度,避免 `OutOfMemoryError`。
- **资源泄漏修复**：`close()` 释放所有 `ByteBuf`，Netty 直接内存可被及时回收。

由于实验重点在于代码级优化与问题修复，未进行大规模性能数据采集；上述效果基于代码路径分析与 JDK/Netty 的通用行为得出。

6 心得体会

本次实验最大的收获是**“先定位确定性开销，再谈优化”**。最初面对 `OutOfMemoryError` 时容易直接调大堆内存，但日志显示 OOM 发生在 `LinkedBlockingQueue.offer`，说明根因是发送端队列无界导致生产速度远大于消费速度。引入有界队列和 `put()` 阻塞后，写入端自然降速，内存问题从机制上被消除。另一方面，接收端的 `Thread.sleep` 忙等待和 `peek()+poll()` 双重操作是典型的“隐蔽开销”，单次影响不大，但在高吞吐下会被放大。通过 `currentBuf` 持有当前块、用 `poll(timeout)` 替代轮询，代码更简洁且语义更清晰。这让我体会到：优化不是盲目改参数，而是理解数据流与控制流，在正确的环节施加背压和减少锁竞争。

7 AI 辅助声明

本报告中的问题定位思路、优化方案设计（包括忙等待替换、`currentBuf` 减少队列操作、有界队列背压机制）在 AI 辅助下完成；具体代码修改、编译调试与实验环境配置由本人独立实施并验证。报告中所有结论均基于对原始代码与运行日志的分析。